1. Introduction to Python

What Python is

Python is a general-purpose programming language designed to be readable and practical. It is used for automation, web applications, data analysis, machine learning, testing, and scripting. Python code is typically executed by an interpreter, which makes it convenient to test small ideas quickly.

Why readability matters

Readable programs are easier to maintain. Use descriptive variable names, keep functions focused, and follow consistent indentation. A program should communicate its intent to another developer, not only to the computer.

Where Python is used

Common applications include processing files, building APIs, automating repetitive tasks, analysing datasets, building prototypes, and connecting to cloud services. Libraries let developers add capabilities without implementing every feature from scratch.

Key idea

A Python program is a set of instructions. The interpreter executes those instructions in order, while control-flow statements and functions determine how the program behaves.

Review task: Explain in your own words what Python is used for.

2. Variables and Data Types

Variables

A variable is a name that refers to a value. For example, student_name = 'Asha' assigns a string to the name student_name. Choose names that explain the purpose of the value.

Common types

The basic built-in types include int for whole numbers, float for decimal numbers, str for text, and bool for True or False values. A value's type determines which operations are meaningful.

Type conversion

Functions such as int(), float(), and str() convert compatible values. Conversion can fail when text does not represent a valid number, so validate external input before relying on it.

Example

age = 21; height = 1.72; course = 'Computer Engineering'; is_student = True. These variables represent an integer, a floating-point number, text, and a Boolean value.

Review task: Identify the type of each value in a small example.

3. Strings and Text Processing

Strings

A string is a sequence of characters written inside single or double quotes. Strings are immutable, which means an operation creates a new string instead of changing the original object.

Useful operations

Use strip() to remove surrounding whitespace, lower() to normalize case, split() to divide text into pieces, and join() to combine pieces. The in operator checks whether a substring occurs inside a string.

Formatting

F-strings make it convenient to insert values into text, such as f'Hello, {name}'. Formatting is useful for readable messages, reports, and application output.

Text cleaning

When processing documents, remove repeated whitespace and empty text carefully. Keep meaningful punctuation and headings when possible because they can help preserve context for search and retrieval.

Review task: Clean a sentence by removing extra whitespace.

4. Lists, Tuples, and Dictionaries

Lists

A list stores an ordered collection of items and can be modified. Use append() to add an item and len() to count items. Indexing begins at zero, so items[0] refers to the first element.

Tuples

A tuple is an ordered collection that is commonly used for values that should remain grouped and unchanged. Tuples can be useful for coordinates or a fixed set of settings.

Dictionaries

A dictionary stores key-value pairs. For example, student = {'name': 'Asha', 'score': 88}. Retrieve a value using its key, such as student['score'].

Choosing a structure

Use a list when order and iteration matter, a tuple for a fixed group of values, and a dictionary when values should be retrieved by meaningful keys.

Review task: Choose a list, tuple, or dictionary for a small use case.

5. Conditions and Loops

Conditional statements

An if statement runs a block when a condition is true. elif checks another condition, and else provides a fallback. Indentation defines the block in Python.

For loops

A for loop iterates over items in a collection. It is useful for processing every file, every record, or every text chunk in a dataset.

While loops

A while loop repeats as long as its condition remains true. Ensure the condition can eventually become false, otherwise the program may run indefinitely.

Loop control

break exits a loop, while continue skips to the next iteration. Use these statements when they make the program's intention clearer.

Review task: Write a loop that processes every item in a list.

6. Functions and Modules

Functions

A function groups reusable instructions under a name. Define one with def, give it parameters when it needs input, and use return to send a result back to the caller.

Small functions

Functions are easier to test when they perform one clear task. For example, a clean_text function should clean text rather than also download files and print a report.

Modules

A module is a Python file that can be imported by another file. The standard library includes modules for paths, dates, JSON, mathematics, and many other tasks.

Main guard

The statement if __name__ == '__main__': runs code only when the file is executed directly. It helps keep reusable functions separate from program startup behavior.

Review task: Design a function that takes input and returns a result.

7. Files and PDF Text Extraction

File paths

A path identifies a file or folder. pathlib.Path provides convenient methods for joining paths, finding files, checking existence, and reading text.

Reading text

Use a context manager such as with open(path, encoding='utf-8') as file to read text files safely. The context manager closes the file even if an error occurs.

PDF extraction

Libraries such as pypdf can extract text from text-based PDF pages. A scanned PDF may contain only images, so OCR may be needed to convert the page images into searchable text.

Document metadata

Keep track of the source filename and page number when extracting text. This makes it possible to show where an answer came from and verify it against the original document.

Review task: Explain why source filenames and page numbers should be preserved.

8. Exceptions and Error Handling

Why errors occur

Programs can encounter missing files, invalid input, network failures, permission errors, and unexpected data. Error handling helps a program report these situations clearly.

Try and except

A try block contains code that may fail, while except handles a specific error. Catch the narrowest useful exception instead of hiding every problem with a broad handler.

Useful messages

A helpful error message states what failed and what the user can do next. For example, a missing-document message can tell the user to place a PDF in the documents folder.

Logging

For larger applications, logging records diagnostic details with levels such as INFO, WARNING, and ERROR. Avoid logging API keys, passwords, or confidential document content.

Review task: Describe a useful error message for a missing PDF.

9. APIs and Environment Variables

What an API is

An application programming interface defines how software components communicate. A web API often accepts a request and returns structured data, commonly in JSON format.

API keys

An API key identifies or authorizes a client. Treat it as a secret: do not place it directly in source code, public screenshots, or a public repository.

Environment variables

Environment variables allow configuration to be supplied outside the program. A local .env file can be loaded during development, but it should be excluded from version control.

Reliability

External APIs can fail because of rate limits, invalid credentials, connectivity problems, or service outages. Display a useful message and avoid printing secrets when handling these failures.

Review task: Explain why API keys should not be committed to Git.

10. Embeddings and Semantic Search

What embeddings represent

An embedding is a numeric vector representing features of text. Texts with related meanings can be close to one another in the embedding space, depending on the model.

Semantic search

Semantic search compares a question embedding with document-chunk embeddings. It can find conceptually related text even when the wording is not identical.

Similarity and distance

Cosine similarity and cosine distance are common measures. Their interpretation depends on the metric and implementation. Always inspect retrieved passages instead of assuming a score proves relevance.

Model consistency

Use the same compatible embedding model for document chunks and user questions. If the embedding model changes, existing vectors may need to be regenerated.

Review task: Describe how semantic search differs from exact keyword matching.

11. Vector Databases

Purpose

A vector database stores embeddings and supports similarity search. It can also store the original text and metadata associated with each vector.

ChromaDB

Chroma supports local persistent collections. A collection can store document text, embeddings, unique identifiers, and metadata such as filename, page, and chunk number.

Identifiers

Each stored record should have a stable unique identifier. Duplicate identifiers can cause insertion problems, so define a consistent ID strategy for pages and chunks.

Persistence

A persistent database saves data to disk between runs. Plan whether ingestion should rebuild the collection or update only new and changed documents.

Review task: Name three pieces of information stored with a vector.

12. Chunking Documents

Why chunking matters

Long documents are divided into smaller passages because retrieval usually works better when each result focuses on a coherent topic. A whole book as one vector may be too broad for a specific question.

Chunk size

A starting point might be a few hundred tokens per chunk, with overlap between adjacent chunks. The best size depends on document structure, embedding model, and question type.

Overlap

Overlap preserves context that might otherwise fall at a boundary. Too much overlap creates redundant results and increases storage, so choose a moderate amount and evaluate it.

Structure-aware splitting

Prefer paragraph, heading, or sentence boundaries when practical. Inspect sample chunks to ensure they are understandable and not cut off in confusing places.

Review task: Explain why chunk overlap can help retrieval.

13. Retrieval-Augmented Generation

RAG workflow

RAG combines retrieval with language generation. First retrieve passages related to the question, then provide those passages to the language model as context for its answer.

Grounded prompts

Tell the model to answer from the supplied context and to say when the answer is missing. Ask it to cite page numbers or source names when those are available in metadata.

Retrieved content is data

Document text may contain misleading or irrelevant instructions. Treat retrieved text as reference material, not as trusted instructions that can override the system's rules.

Limits

RAG does not guarantee correctness. Retrieval may miss the right passage, documents may be outdated, and a model may still misinterpret context. Evaluation and careful prompt design are important.

Review task: Describe what the model should do when context lacks an answer.

14. Evaluating a RAG System

Retrieval evaluation

Create a set of questions with known supporting passages. Check whether the correct passage appears among the top results. This evaluates retrieval separately from answer generation.

Answer evaluation

Compare the generated answer with the source document. Check factual correctness, whether the evidence supports the answer, and whether the system admits when information is unavailable.

RAG versus no RAG

Ask the same document-specific question with and without retrieved context. A strong demonstration shows the RAG answer matching a real source passage while the plain model lacks that private information.

Record evidence

Keep the question, retrieved chunks, source pages, RAG answer, and plain-model answer in a README. Report actual outputs and do not invent successful results.

Review task: Design one question to evaluate retrieval quality.

15. Security, Deployment, and Next Steps

Protect secrets

Keep API keys in environment variables and exclude .env from Git. If a key is accidentally published, revoke it and create a replacement.

Document privacy

Only ingest documents you are allowed to use. Before publishing a repository, verify that sample PDFs and extracted text do not contain private, personal, or restricted information.

Reproducibility

Document Python requirements, setup steps, the embedding model, chunk size, overlap, retrieval count, and how to run the application.

Future improvements

Useful next steps include a web interface, support for Word documents, OCR for scanned PDFs, better chunking, incremental ingestion, reranking, and automated retrieval tests.

Conclusion

A reliable RAG application is more than a chatbot connected to a vector database. It needs useful documents, sensible chunking, relevant retrieval, grounded generation, honest evaluation, and safe handling of data.

Review task: List two security checks to perform before publishing a repository.